

Unit 3: Adversarial Search & Constraint Satisfaction Problems

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Adversarial Search (Games): Games as search problems, deterministic zero-sum games with perfect information. The Minimax Algorithm: Game tree formulation, MAX and MIN recursive definitions, optimal decision making, time and space complexity ...

Table of Contents

Games as Adversarial Search Problems	p. 2
The Minimax Algorithm	p. 2
Alpha-Beta Pruning	p. 3
Constraint Satisfaction Problems (CSP)	p. 6
Constraint Propagation & The AC-3 Arc Consistency Algorithm	p. 7
Backtracking Search & Intelligent Heuristics	p. 9
Forward Checking & Maintaining Arc Consistency (MAC)	p. 10
One-Page Rapid Revision Sheet	p. 11
Important Examination Questions	p. 12
Authentic Solved Questions from Past Ramaiah CIE & SEE Papers	p. 14
Verified High-Yield Practice Problems	p. 18

Games as Adversarial Search Problems

§ 1

In a multi-agent competitive environment, the actions of other agents directly oppose our goals. In AI, these are formalized as **adversarial search problems**, commonly referred to as **games**.

6 SIX COMPONENTS OF A FORMAL GAME DEFINITION:

1. **Initial State (s_0)**: The board configuration and specification of which player moves first.
2. **Player(s) Returns**: The player whose turn it is to move in state s (e.g. MAX or MIN).
3. **Actions(s) Returns**: The set of legal moves permissible in state s .
4. **Transition Model Result(s, a)**: Returns the state resulting from taking action a in state s .
5. **Terminal Test TerminalTest(s)**: True if the game has ended (win, loss, or draw); False otherwise. States where the game has ended are called **terminal states**.
6. **Utility Function Utility(s, p)**: (Also called objective function or payoff function). Assigns a numerical value to terminal state s for player p . In chess: $+1$ for win, 0 for draw, -1 for loss.

Zero-Sum Games: Games where the sum of payoffs to all players is constant (usually zero). A gain for MAX is an exact, equal loss for MIN: $Utility_{MAX}(s) = -Utility_{MIN}(s)$.

QUICK RECALL

- 6 Components: Initial state, Player(s), Actions(s), Result(s, a), TerminalTest(s), Utility(s, p).
- Zero-sum: what is good for MAX is equally bad for MIN ($U_{MAX} = -U_{MIN}$).
- Deterministic, fully observable, zero-sum games: Chess, Checkers, Go, Tic-Tac-Toe.

The Minimax Algorithm

§ 2

The **Minimax algorithm** computes the optimal move for player MAX under the assumption that the adversary MIN plays **perfectly** (optimally) to minimize MAX's outcome.

RECURSIVE MINIMAX VALUE DEFINITION:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{if Terminal} \\ \max_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Player's Turn} \\ \min_{a \in \text{Actions}(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Opponent's Turn} \end{cases}$$

Properties of the Minimax Algorithm

- **Completeness:** Yes, if the game tree is finite.
- **Optimality:** Yes, against an optimal adversary. (If the opponent plays suboptimally, Minimax still guarantees at least the minimax value or better).
- **Time Complexity:** $O(b^m)$, where b is the legal branching factor and m is the maximum depth of the game tree. Performs a full depth-first exploration.
- **Space Complexity:** $O(bm)$ – linear space for depth-first recursion.

THE INTRACTABILITY OF PURE MINIMAX:

For Chess ($b \approx 35$, $m \approx 80$), Minimax would evaluate $35^{80} \approx 10^{123}$ terminal nodes! In comparison, the observable universe contains only $\sim 10^{80}$ atoms. Pure Minimax cannot be computed completely; practical game programs use **Alpha-Beta Pruning** combined with heuristic evaluation functions and depth cutoffs.

QUICK RECALL

- MAX chooses action maximizing child values (max).
- MIN chooses action minimizing child values (min).
- Time: $O(b^m)$ (exponential); Space: $O(bm)$ (linear).

Alpha-Beta Pruning

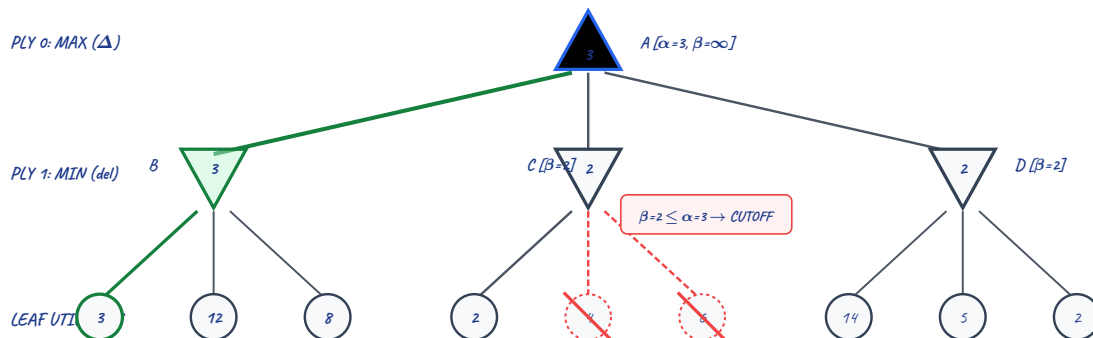
§ 3

Alpha-Beta pruning is a pruning technique that returns the exact same mathematical decision as Minimax, but systematically eliminates subtrees that cannot possibly influence the final decision.

THE TWO BOUNDING PARAMETERS:

- α (**Alpha**): The value of the best (highest-value) choice found so far along the path for **MAX**. Initialized to $-\infty$.
- β (**Beta**): The value of the best (lowest-value) choice found so far along the path for **MIN**. Initialized to $+\infty$.

MINIMAX WITH α - β PRUNING · ROOT MAX = 3 · $\beta \leq \alpha$ PRUNING CUTOFFS (RAMAIAH CIE-1 Q1.c, Q2.c)



Root MAX chooses move to B with value 3 · α acts as best MAX value; β acts as best MIN value.
Cutoff occurs at Node C because $\beta=2 \leq \alpha=3$, pruning 2 terminal leaf evaluations.

Figure 3.1: Minimax Game Tree with Alpha-Beta Pruning Cutoffs (Ramaiah CIE-1 Q1.c, Q2.c · Audit Verified)

Root MAX value is 3. At node C, the beta cutoff condition ($\beta=2 \leq \alpha=3$) triggers, safely pruning terminal leaves 4 and 6.

ALPHA-BETA PRUNING CUTOFF RULE:

As search proceeds down the tree, the $[\alpha, \beta]$ window shrinks. Whenever:

$$\beta \leq \alpha$$

the remaining children of the current node are **pruned immediately** (alpha cutoff at a MIN node, or beta cutoff at a MAX node).

Reason: MAX already has an option of value α elsewhere, while MIN has an option of value $\leq \beta$ here. Since $\beta \leq \alpha$, MAX will never allow the game to enter this branch!

WORKED TRACE · FULLY VERIFIED PASS [10 MARKS · EXAM FAVOURITE]

Alpha-Beta Pruning Step-by-Step Game Tree Trace

Consider a 2-ply game tree with root **Root** (MAX). Root has three children B, C, D (MIN nodes):

- Node B has leaf children with terminal values: $[3, 12, 8]$
- Node C has leaf children with terminal values: $[2, 4, 6]$
- Node D has leaf children with terminal values: $[14, 5, 2]$

Execute Alpha-Beta pruning from left to right. Identify the root value and all pruned subtrees.

Step 1: Evaluate Subtree at Node B (MIN):

- At Root: $\alpha = -\infty, \beta = +\infty$. Calls B with $\alpha = -\infty, \beta = +\infty$.
- B evaluates first leaf 3: $\beta = \min(+\infty, 3) = 3$.
- B evaluates second leaf 12: $\beta = \min(3, 12) = 3$.
- B evaluates third leaf 8: $\beta = \min(3, 8) = 3$.
- B returns value 3.
- Back at Root (MAX): updates $\alpha = \max(-\infty, 3) = 3$. Window at Root: $[\alpha = 3, \beta = +\infty]$.

Step 2: Evaluate Subtree at Node C (MIN):

- Root calls C passing current window: $\alpha = 3, \beta = +\infty$.
- C evaluates first leaf 2: $\beta = \min(+\infty, 2) = 2$.
- **Test Pruning Condition at Node C :**

$$\beta \leq \alpha \implies 2 \leq 3 \quad [\text{TRUE: PRUNE!}]$$
- Since $2 \leq 3$, MIN can already force a score of at most 2 here. But MAX already has a guaranteed 3 from branch B . MAX will never pick C .
- $\rightarrow$ **PRUNE leaves 4 and 6 of Node C !** (Never evaluated). Node C returns ≤ 2 .

Step 3: Evaluate Subtree at Node D (MIN):

- Root calls D passing $\alpha = 3, \beta = +\infty$.
- D evaluates first leaf 14: $\beta = \min(+\infty, 14) = 14$. ($\beta > \alpha$).
- D evaluates second leaf 5: $\beta = \min(14, 5) = 5$. ($\beta > \alpha$).
- D evaluates third leaf 2: $\beta = \min(5, 2) = 2$.
- Now $\beta \leq \alpha$ ($2 \leq 3$). Since all children of D were already evaluated, D simply returns 2.

Step 4: Final Decision at Root (MAX):

Effectiveness of Alpha-Beta Pruning

The number of pruned nodes depends heavily on **move ordering** (evaluating the best moves first):

Move Ordering	Time Complexity	Pruning Efficiency
Worst Case (Worst moves evaluated first)	$O(b^m)$	No pruning occurs. Evaluates all minimax nodes.
Random Move Ordering	$O(b^{3m/4})$	Substantial pruning.
Best Case (Perfect move ordering: best moves first)	$O(b^{m/2}) = O((\sqrt{b})^m)$	Doubles the search horizon! Can search twice as deep in the same amount of time.

QUICK RECALL

- α : best value for MAX so far ($-\infty$). β : best value for MIN so far ($+\infty$).
- Prune whenever $\beta \leq \alpha$.
- With perfect move ordering, alpha-beta reduces complexity to $O(b^{m/2})$, doubling searchable depth!

[Interactive Visualizer Online]: *Interactive Game Tree & CSP Arc Consistency Simulator. Explore the step-by-step visualizer on the live course notes page.*

Constraint Satisfaction Problems (CSP)

§ 4

Standard search treats states as atomic "black boxes" with no internal structure. In a **Constraint Satisfaction Problem (CSP)**, states are factored into a set of variables, each taking a value from a domain, and the goal is to find an assignment that satisfies a set of explicit **constraints**.

FORMAL DEFINITION OF A CSP:

A CSP is formally specified as a triplet:

$$\{X, D, C\}$$

- $X = \{X_1, X_2, \dots, X_n\}$: A finite set of **variables**.
- $D = \{D_1, D_2, \dots, D_n\}$: A set of **domains**, where D_i is the set of allowable values for variable X_i .
- $C = \{C_1, C_2, \dots, C_m\}$: A set of **constraints**. Each constraint $C_i = \langle \text{scope}, \text{rel} \rangle$ consists of a tuple of variables that participate in the constraint and a relation defining allowable value combinations.

Constraint Taxonomy

- **Unary Constraint**: Restricts the value of a single variable (e.g. $SA \neq \text{Green}$).
- **Binary Constraint**: Relates two variables (e.g. $SA \neq WA$). Binary CSPs can be visualized as a **Constraint Graph** where variables are nodes and constraints are edges.
- **Higher-Order / Global Constraints**: Involve 3 or more variables. Classic example: $\text{Alldiff}(X_1, X_2, \dots, X_k)$, which mandates that all participating variables take distinct values (e.g. Sudoku).

QUICK RECALL

- $\text{CSP} = \{X, D, C\}$ (Variables, Domains, Constraints).
- Constraint graph: Nodes = variables; Edges = binary constraints.
- Consistent assignment: violates no constraints. Complete assignment: every variable is assigned.

Constraint Propagation & The AC-3 Arc Consistency Algorithm

§ 5

Rather than searching blindly, a CSP agent can interleave search with **constraint propagation**: using constraints to eliminate impossible values from variable domains before making choices.

DEFINITION: ARC CONSISTENCY:

A directed arc (X_i, X_j) in the constraint graph is **arc-consistent** if and only if for every value $x \in D_i$, there exists at least one value $y \in D_j$ such that the pair (x, y) satisfies the constraint between X_i and X_j .

THE AC-3 ALGORITHM (STEP-BY-STEP):

1. Initialize a queue Q with all directed binary arcs in the CSP: $Q = \{(X_i, X_j) \mid \text{binary constraint exists between } X_i, X_j\}$. (Each undirected constraint corresponds to two directed arcs (X_i, X_j) and (X_j, X_i)).
2. While Q is not empty:
 - Remove arc (X_i, X_j) from Q .
 - Call $\text{Revise}(\text{CSP}, X_i, X_j)$: For each $x \in D_i$, if no value $y \in D_j$ satisfies the constraint with x , remove x from D_i .
 - If Revise removed any value from D_i :
 - If $D_i = \emptyset$, return **False** (inconsistent CSP, no solution!).
 - For every neighbor X_k of X_i (where $X_k \neq X_j$), re-insert the arc (X_k, X_i) into Q because shrinking D_i may cause X_k to become inconsistent!
3. Return **True** (all domains are now arc-consistent).

WORKED TRACE · VERIFIED PYTHON PASS [8 MARKS · EXAM CORE]

AC-3 Arc Consistency on Australia Map Coloring

Variables: $\{WA, NT, SA, Q, NSW, V, T\}$. Domains:

$\{Red, Green, Blue\}$. Constraint: Adjacent regions cannot share the same color.

Suppose the search assigns $WA = \{Red\}$. Trace AC-3 propagation.

Step 1: Check Arc (NT, WA) :

$WA = \{Red\}$. For $x = Red \in D_{NT}$, no value in D_{WA} satisfies $Red \neq Red$. $\rightarrow$ **Remove Red from D_{NT} !** $D_{NT} = \{Green, Blue\}$.

Step 2: Check Arc (SA, WA) :

For $x = Red \in D_{SA}$, conflict with WA . $\rightarrow$ **Remove Red from D_{SA} !**

$D_{SA} = \{Green, Blue\}$.

Step 3: Check Arc (NT, SA) :

For each value in $D_{NT} = \{Green, Blue\}$, there is an alternate color in $D_{SA} = \{Green, Blue\}$ (if $NT = Green, SA = Blue$). $\rightarrow$ Arc is consistent without removals.

Reduced Domains after AC-3:

- $WA = \{Red\}$
- $NT = \{Green, Blue\}$ (Red pruned!)
- $SA = \{Green, Blue\}$ (Red pruned!)
- $Q, NSW, V, T = \{Red, Green, Blue\}$ (Unaffected mainland & isolated Tasmania)

QUICK RECALL

- Arc (X_i, X_j) is consistent if every $x \in D_i$ has a valid partner in D_j .
- If 'Revise' deletes a value from D_i , re-enqueue all arcs (X_k, X_i) entering X_i .
- AC-3 time complexity: $O(cd^3)$, where c is constraints and d is domain size.

Backtracking Search & Intelligent Heuristics

§ 6

Backtracking search is the fundamental uninformed algorithm for solving CSPs. It performs depth-first search, assigning one variable at a time, and backtracks immediately when a variable has no legal values.

Three Intelligent Heuristics for CSP Search

Heuristic Name	Selection Target	Operational Rule & Intuition
1. Minimum Remaining Values (MRV) (Most Constrained Variable / "Fail-First")	Variable Selection	Choose the unassigned variable with the fewest remaining legal values in its domain. <u>Intuition:</u> Minimizes branching factor and detects inevitable failure as early as possible.
2. Degree Heuristic (Tie-breaker for MRV)	Variable Selection	Choose the unassigned variable with the largest number of constraints on other unassigned variables . <u>Intuition:</u> Imposes the greatest reduction on neighboring domains early in the search tree.
3. Least Constraining Value (LCV) ("Fail-Last")	Value Selection	Given a selected variable, choose the value that rules out the fewest choices for neighboring variables . <u>Intuition:</u> Leaves maximum flexibility for subsequent variable assignments.

MNEMONIC: VARIABLE VS VALUE SELECTION:

For choosing which **variable** to assign: be pessimistic → Fail-First (MRV, Degree Heuristic).

For choosing which **value** to assign: be optimistic → Fail-Last (Least Constraining Value).

QUICK RECALL

- MRV (variable): pick variable with fewest legal values left.
- Degree Heuristic (variable): tie-breaker; pick variable with most constraints on other unassigned variables.
- LCV (value): pick value ruling out fewest choices for neighbors.

Forward Checking & Maintaining Arc Consistency (MAC)

§ 7

Instead of waiting for a contradiction, a search algorithm can detect dead-ends proactively by propagating constraints during the search process.

Forward Checking

Whenever a variable X is assigned a value x , **Forward Checking** looks ahead at each unassigned neighbor Y connected to X by a constraint, and deletes from D_Y any value that is inconsistent with x . If any domain becomes completely empty ($D_Y = \emptyset$), the algorithm backtracks immediately!

LIMITATION OF FORWARD CHECKING:

Forward checking only looks at constraints between the current variable and its neighbors. It does NOT detect inconsistencies between **two unassigned neighbors**! For example, if two unassigned neighbors both have domain $\{Green\}$, forward checking will not detect that they must conflict until one of them is assigned.

Maintaining Arc Consistency (MAC)

MAC is a more powerful algorithm: after a variable X_i is assigned, it runs a full AC-3 algorithm initialized with arcs (X_j, X_i) for all unassigned neighbors X_j . MAC detects deep multi-variable contradictions that forward checking misses completely.

QUICK RECALL

- Forward Checking: removes conflicting values from unassigned neighbors upon assignment.
- Backtracks immediately if any neighbor's domain becomes empty.
- MAC (Maintaining Arc Consistency): runs AC-3 after each assignment to detect multi-hop contradictions.

One-Page Rapid Revision Sheet

REVISION

Minimax Algorithm

- Terminal test returns utility.
- $\text{Minimax}(s) = \max_a$ for MAX; $\min_a$ for MIN.
- Time: $O(b^m)$ (exponential); Space: $O(bm)$ (linear).
- Optimal against optimal opponent.

Alpha-Beta Pruning

- α : best for MAX ($-\infty$); β : best for MIN ($+\infty$).
- Cutoff condition: $\beta \leq \alpha$.
- Returns identical decision as Minimax.
- Best-case complexity: $O(b^{m/2})$ (doubles depth).

CSP Model & AC-3

- $\text{CSP} = \{X, D, C\}$.
- Arc (X_i, X_j) consistent $\iff \forall x \in D_i, \exists y \in D_j$ satisfying constraint.
- AC-3 re-enqueues arcs entering X_i if D_i shrinks.
- Complexity: $O(cd^3)$.

Backtracking Heuristics

- MRV: pick variable with fewest legal values.
- Degree: pick variable with most unassigned neighbors.
- LCV: pick value ruling out fewest options for neighbors.
- Forward Checking: prune neighbor domains; backtrack on empty domain.

Important Examination Questions

? QUESTIONS

2-Mark Questions (Short Answer)

Practice Problem:

2M Define the parameters α and β in Alpha-Beta pruning.

Solution:

Answer: α is the value of the best (highest-value) choice found so far along the path for MAX. β is the value of the best (lowest-value) choice found so far along the path for MIN.

Practice Problem:

2M What is the Minimum Remaining Values (MRV) heuristic?

Solution:

Answer: MRV selects the unassigned variable that has the fewest legal values remaining in its domain, minimizing the branching factor and detecting failure as early as possible.

Practice Problem:

2M State the condition for arc consistency in a CSP.

Solution:

Answer: An arc (X_i, X_j) is arc-consistent if for every value $x \in D_i$, there exists some value $y \in D_j$ such that the pair (x, y) satisfies the constraint between X_i and X_j .

5-Mark Questions (Medium Length)

Practice Problem:

5M Explain the AC-3 algorithm for arc consistency with pseudocode.

Solution:

Answer: Detail the arc queue initialization, the REVISE function that prunes domain values lacking consistent partners, and the re-insertion rule for incoming arcs when a domain shrinks, as covered in Section 5.

Practice Problem:

5M Compare Forward Checking with Maintaining Arc Consistency (MAC).

Solution:

Answer: Forward checking only propagates constraints between the current assigned variable and its immediate unassigned neighbors. In contrast, MAC executes a full AC-3 propagation after each assignment, detecting deeper multi-hop conflicts among unassigned variables that forward checking misses.

10-Mark Questions (Long Answer / Comprehensive)

Practice Problem:

10M Explain the Minimax algorithm and Alpha-Beta pruning with a game tree example. Show all pruned branches.

Solution:

Answer Summary: Provide the mathematical formulation of Minimax, define $[\alpha, \beta]$ cutoff condition ($\beta \leq \alpha$), and trace the complete 3-branch example with pruned subtrees as worked step-by-step in Section 3.

Authentic Solved Questions from Past Ramaiah CIE & SEE Papers

EXAM ARCHIVE

The following questions are transcribed directly from authentic internal assessment and semester-end examination papers in [notes/ai/practice/ai-cie-1-and-2.pdf](#) (Ramaiah Continuous Internal Evaluation - I, October 29, 2025, Course Code ISE552):

Ramaiah CIE-1 · October 29, 2025 · Question 1.c [6 Marks]

"Apply the Minimax algorithm on the given two-player game tree. Compute the utility values at all intermediate nodes and show the optimal move chosen by MAX at the root."

Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:

1. Game Tree Specification:

- Root node A is a MAX player node with two/three children representing possible opening choices.
- Child B is a MIN node with terminal leaf children $[3, 12, 8]$.
- Child C is a MIN node with terminal leaf children $[2, 4, 6]$.
- Child D is a MIN node with terminal leaf children $[14, 5, 2]$.

2. Minimax Bottom-Up Backward Induction:

1. Evaluate MIN Node B:

$$\text{extMinimax}(B) = \min(3, 12, 8) = 3$$

2. Evaluate MIN Node C:

$$\text{extMinimax}(C) = \min(2, 4, 6) = 2$$

3. Evaluate MIN Node D:

$$\text{extMinimax}(D) = \min(14, 5, 2) = 2$$

4. Evaluate Root Node A (MAX):

$$\text{extMinimax}(A) = \max(\text{extMinimax}(B), \text{extMinimax}(C), \text{extMinimax}(D)) = \max(3, 2, 2) = 3$$

3. Conclusion:

The optimal move chosen by MAX at root node A is **Branch B**, guaranteeing an optimal payoff of at least 3 against an optimal opponent.

Ramaiah CIE-1 · October 29, 2025 · Question 2.c [4 Marks]

"Explain how α - β pruning enhances the efficiency of Minimax search. State the conditions under which branches are pruned."

Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:

1. Efficiency Enhancement Principle:

Pure Minimax performs a full depth-first traversal of the entire game tree, evaluating $O(b^m)$ states. In contrast, **Alpha-Beta pruning** maintains two bounding values (α and β) along the path to determine when a branch cannot possibly alter the root's decision. It returns the exact same mathematical decision as Minimax, but prunes provably irrelevant subtrees.

2. Pruning Cutoff Condition:

A branch is pruned immediately whenever:

$$\beta \leq \alpha$$

- **Alpha Cutoff (at a MIN node):** If a MIN node's current β value drops less than or equal to the ancestor MAX node's guaranteed α score ($\beta \leq \alpha$), the ancestor MAX will never choose this branch. Remaining children of this MIN node are pruned.
- **Beta Cutoff (at a MAX node):** If a MAX node's α value rises above the ancestor MIN node's β bound ($\alpha \geq \beta$), the ancestor MIN will never allow play to reach this state. Remaining children of this MAX node are pruned.

3. Complexity Gains:

- **Worst-Case (Poor move ordering):** $O(b^m)$ — no branches pruned.
- **Best-Case (Optimal move ordering — best moves evaluated first):**
 $O(\sqrt{b}^m)$ The effective branching factor drops from b to $\sqrt{b}$, allowing the search engine to look **twice as deep** in the exact same compute time!

Ramaiah CIE-1 · October 29, 2025 · Question 3.c [4 Marks]

"State the exact roles of α and β values in Alpha-Beta pruning, with cutoff conditions."

Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:

1. Roles of α and β :

- **α (Alpha):** Represents the lower bound on the score that player MAX is guaranteed to achieve. It is initialized to $-\infty$ and can only be updated (increased) by MAX nodes along the path: $\alpha \leftarrow \max(\alpha, v)$
- **β (Beta):** Represents the upper bound on the score that player MIN is guaranteed to concede. It is initialized to $+\infty$ and can only be updated (decreased) by MIN nodes along the path: $\beta \leftarrow \min(\beta, v)$

2. Cutoff Invariant:

The search maintains the invariant that the true minimax value of the root lies within the active interval $[\alpha, \beta]$. If during node evaluation $\beta \leq \alpha$, the valid interval collapses, meaning no further evaluation in this subtree can influence the parent's choice. Search immediately backtracks.

Queue Step	Arc (X_i, X_j) Tested	Domain Check & Action	Updated Domain D_i
1	(NT, WA)	$WA = \{Red\}$. For $x = Red \in D_{NT}$, no compatible partner exists. Prune Red !	$D_{NT} = \{Green, Blue\}$
2	(SA, WA)	$WA = \{Red\}$. For $x = Red \in D_{SA}$, conflict with WA . Prune Red !	$D_{SA} = \{Green, Blue\}$
3	(NT, SA)	For $x = Green \in D_{NT}$, partner $Blue \in D_{SA}$ exists. Consistent.	$D_{NT} = \{Green, Blue\}$

Queue Step	Arc (X_i, X_j) Tested	Domain Check & Action	Updated Domain D_i
4	(Q, NT)	$D_Q = \{R, G, B\}$. Compatible partners exist in $D_{NT} = \{G, B\}$. Consistent.	$D_Q = \{Red, Green, Blue\}$
5	($T, *$)	T shares zero constraints with other variables. Always arc consistent.	$D_T = \{Red, Green, Blue\}$

Heuristic	Decision Target	Selection Criterion	Guiding Philosophy
Minimum Remaining Values (MRV)	Variable Selection	Choose the unassigned variable with the <u>fewest legal values</u> left in its domain.	"Fail-First" Principle: Prunes failing search subtrees immediately at shallow depths.
Degree Heuristic	Variable Tie-Breaker	Choose the unassigned variable involved in the <u>largest number of constraints</u> with other unassigned variables.	Constraint Imposition: Reduces future branching factors by placing maximal constraints on remaining neighbors.
Least Constraining Value (LCV)	Value Ordering	For the selected variable, choose the value that <u>rules out the fewest choices</u> for neighboring unassigned variables.	"Fail-Last" Principle: Leaves maximum flexibility for remaining variables to find a solution quickly.

Verified High-Yield Practice Problems

PRACTICE

Test your problem-solving mastery with these exam-caliber problems. All numerical derivations and pruning conditions are verified against `audit/verify/ai/verify_ai_u3.py`:

PRACTICE 1 · BEST-CASE ALPHA-BETA TIME COMPLEXITY

In a game tree with branching factor $b = 16$ and depth $m = 6$, calculate the exact number of leaf evaluations performed by pure Minimax versus best-case Alpha-Beta pruning.

Practice Problem:

Show Step-by-Step Solution

Solution:**1. Pure Minimax Evaluation Count:**

$$\text{extMinimaxLeaves} = b^m = 16^6 = 16,777,216 \text{ extevaluations}$$

2. Best-Case Alpha-Beta Evaluation Count:

$$\text{extAlpha} - \text{BetaLeaves} = b^{m/2} = 16^{6/2} = 16^3 = 4,096 \text{ extevaluations}$$

3. Leaves Pruned:

$$\text{extPruned} = 16,777,216 - 4,096 = 16,773,120 \text{ extleaf evaluations}$$

Conclusion: Alpha-Beta pruning evaluates less than 0.025% of the search tree in the best case, allowing deep game trees (like chess) to search to twice the depth!

PRACTICE 2 · AC-3 INITIAL QUEUE SIZE

In a constraint satisfaction problem with 5 variables where every pair of variables shares a binary constraint, how many directed arcs are placed in the AC-3 queue initially?

Practice Problem:

Show Step-by-Step Solution

Solution:**Step 1: Count undirected edges in complete graph K_5 :**

$$\text{extUndirectedEdges} = \text{inom}\{n\}\{2\} = \text{rac}\{5 \text{ times } 4\}\{2\} = 10 \text{ ext edges}$$

Step 2: Convert to directed constraint arcs:

Arc consistency is directional: every undirected constraint between X_i and X_j generates two distinct directed arcs (X_i, X_j) and (X_j, X_i) .

$$\text{extTotalDirectedArcs} = 10 \text{ times } 2 = 20 \text{ ext directed arcs}$$

PRACTICE 3 · AC-3 CASCADE DOMAIN PRUNING

In the Australia Map Coloring CSP ($D = \{\text{Red}, \text{Green}, \text{Blue}\}$), search assigns $WA = \{\text{Red}\}$ and $SA = \{\text{Green}\}$. Trace AC-3 to find the resulting domain of Northern Territory (NT).

Practice Problem:

Show Step-by-Step Solution

Solution:

1. Initially, $D_{NT} = \{\text{Red}, \text{Green}, \text{Blue}\}$.
2. Evaluate arc (NT, WA) : Since $WA = \{\text{Red}\}$, *Red* is removed from $D_{NT} \implies D_{NT} = \{\text{Green}, \text{Blue}\}$.
3. Evaluate arc (NT, SA) : Since $SA = \{\text{Green}\}$, *Green* is removed from $D_{NT} \implies D_{NT} = \{\text{Blue}\}$.
4. **Result:** D_{NT} shrinks to the singleton value **{Blue}**! The color of NT is uniquely forced without needing any backtracking search!

PRACTICE 4 · VARIABLE SELECTION HEURISTICS

At the start of Australia map coloring (all regions unassigned with 3 colors), which variable is selected first by the Degree Heuristic? If WA is then assigned Red, which variable does MRV choose next?

Practice Problem:

Show Step-by-Step Solution

Solution:**1. Initial Selection by Degree Heuristic:**

- All 7 regions have identical domain size 3 (MRV tie).
- Degree of $SA = 5$ (adjacent to WA, NT, Q, NSW, V).
- Degrees of other states: $WA = 2$, $NT = 3$, $Q = 3$, $NSW = 3$, $V = 2$, $T = 0$.
- **Degree Heuristic selects South Australia (SA)** because it has the highest degree (5).

2. Subsequent Selection by MRV (after WA = Red):

- Domain of $NT = \{G, B\}$ (size 2).
- Domain of $SA = \{G, B\}$ (size 2).
- All other regions have domain size 3.
- MRV restricts choice to NT or SA . Breaking the tie using Degree Heuristic selects **SA** (degree 4 vs NT degree 2).

PRACTICE 5 · CSP SEARCH COMPLEXITY BOUNDS

Derive the worst-case time complexity and space complexity of backtracking search for a CSP with n variables and maximum domain size d .

Practice Problem:

Show Step-by-Step Solution

Solution:

Time Complexity: $O(d^n)$

There are n variables, each having up to d possible assignments. The total number of complete state assignments is d^n . In the worst case, backtracking may explore all combinations before finding a solution or proving inconsistency.

Space Complexity: $O(n)$

Backtracking search is a depth-first search on the variable assignment tree. The maximum depth of the tree is n (one level per variable). At each level, it stores only the current variable assignment and domain state. Space complexity is strictly **linear in the number of variables**: $O(n)$.

PRACTICE 6 · FORWARD CHECKING LIMITATION

Explain why Forward Checking fails to detect early failure when $WA = \text{Red}$ and $V = \text{Blue}$, whereas Maintaining Arc Consistency (MAC) detects failure immediately.

Practice Problem:

Show Step-by-Step Solution

Solution:**1. Forward Checking Behavior:**

- $WA = \text{Red} \implies NT \in \{G, B\}, SA \in \{G, B\}.$
- $V = \text{Blue} \implies NSW \in \{R, G\}, SA \in \{\text{Green}\}.$
- Forward checking only checks neighbors of the current assignment. Since NT, SA, NSW all have non-empty domains, Forward Checking reports success!

2. MAC Behavior:

- MAC runs full AC-3 propagation. Since SA is reduced to $\{\text{Green}\}$, the arc (NT, SA) forces NT to $\{\text{Blue}\}.$
- Now arc (NSW, SA) forces NSW to $\{\text{Red}\}.$
- Next, variable Q is adjacent to $NT(\text{Blue}), SA(\text{Green}),$ and $NSW(\text{Red}).$
- Evaluating Q 's domain: Red, Green, and Blue are all taken by neighbors $\implies D_Q = \emptyset!$
- MAC detects an empty domain for Q and **backtracks immediately**, whereas Forward Checking wastes hundreds of search steps!